

TRAVAUX PRATIQUES

SmartGrid

Smart Grid - Big Data et Intelligence Artificielle

Prévision de consommation énergétique et détection d'anomalies

Pipeline SmartGrid: données, ingestion, ML et monitoring

Etudiant

Elwalid Aboulaakoul
Ahmed Benahmed

Encadrement

Pr. Hrimech

Date du TP

1 juin 2026

Annee universitaire

2025/2026

Rapport de projet avec captures locales, métriques ML et preuves d'exécution.

Résumé

Ce rapport présente un pipeline SmartGrid pour la prévision de consommation énergétique et la détection d'anomalies dans des séries de compteurs intelligents. L'objectif est de transformer des données hétérogènes issues de plusieurs sources en un flux exploitable: ingestion, stockage temporel, préparation Machine Learning, prévision, détection d'écarts et visualisation opérationnelle.

La partie IA est présentée comme un benchmark reproductible plutôt qu'un résultat isolé. Les séries sont ramenées à une cadence de 30 minutes, séparées naturellement par source, puis évaluées avec une validation rolling-origin sur un horizon de 24 heures, soit 48 pas. Prophet reste le modèle interprétable principal, mais il est comparé à SeasonalNaive et LightGBM; le LSTM Autoencoder est conservé comme baseline de recherche pour les anomalies séquentielles.

Les anomalies sont détectées à partir du résidu de prévision: $\text{réel} - \hat{y}$, puis seuil robuste médiane/MAD. Le scoring distingue les lignes individuelles et les événements opérationnels. Pour éviter qu'une seule fluctuation transforme un segment en alerte, un événement non ponctuel doit accumuler $k = 5$ flags; les pics et drops ponctuels gardent $k = 1$.

La démonstration locale valide une chaîne reproductible: génération et préparation de données, prévision de référence, injection d'anomalies labellisées, évaluation précision/rappel, chargement dans TimescaleDB, exposition Grafana et supervision Prometheus/Kafka. Le rapport se concentre donc sur la logique IA et sur les preuves visibles nécessaires pour une soutenance technique fiable.

Contexte et objectifs

Les réseaux électriques intelligents exigent une observation fine de la demande afin d'anticiper les pics, d'identifier les comportements inhabituels et de fournir aux opérateurs une lecture claire de l'état du système. Les compteurs produisent des séries temporelles volumineuses et bruitées; une simple base relationnelle ou un graphe isolé ne suffit pas. Il faut une chaîne complète, depuis l'arrivée des mesures jusqu'à la décision.

Les objectifs du projet sont:

- unifier les données de consommation dans un schéma commun;
- conserver les mesures et agrégats dans une base temporelle adaptée;
- organiser les séries par source de données;
- produire des prévisions court terme et tendance;
- signaler les écarts anormaux de manière interprétable;
- fournir un tableau de bord lisible pour le responsable énergie.

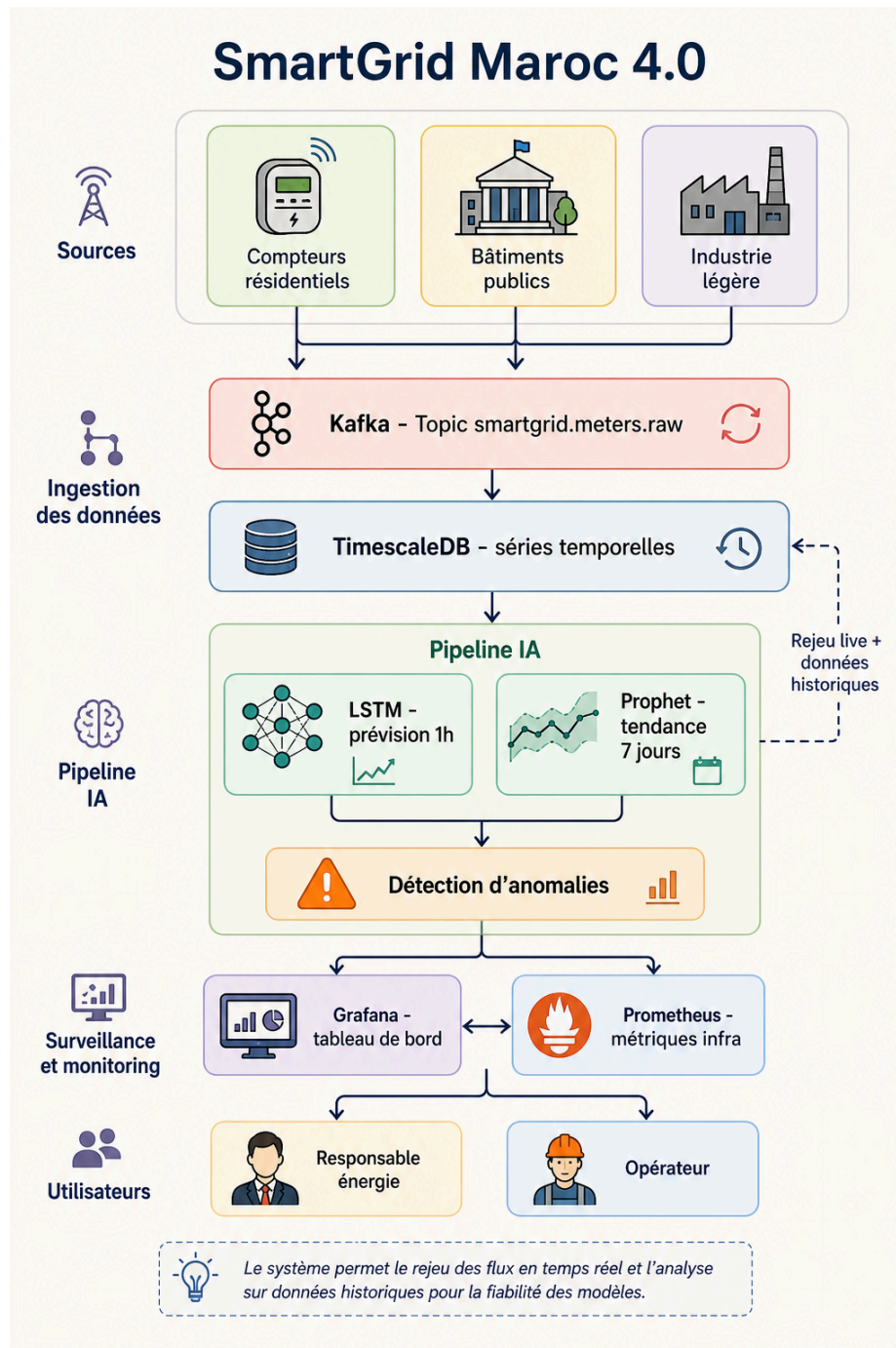


Figure 1: Architecture SmartGrid proposée: sources de mesure, ingestion Kafka, stockage TimescaleDB, couche IA Prophet/LightGBM/LSTM Autoencoder, détection d'anomalies, puis supervision Grafana et Prometheus. La figure résume le rôle de chaque brique avant d'entrer dans les détails techniques.

Sources de données et préparation

Le projet documente quatre familles de données. Morocco High-Resolution Smart Meters donne le contexte marocain et peut nécessiter une conversion depuis l'intensité électrique vers le kWh. London Smart Meters fournit des profils résidentiels riches à granularité demi-horaire. UCI Household Power sert de jeu plus léger pour les tests rapides. Nigeria Smart Meter ajoute une validation africaine complémentaire avec un comportement de consommation différent.

Toutes les sources sont ramenées vers le même format:

```
time, meter_id, kwh, is_anomaly, source
```

Cette normalisation est importante: le modèle et le dashboard ne doivent pas dépendre de la forme originale d'un fichier London, Morocco, Nigeria ou UCI. Le champ source conserve l'origine de la série et permet d'éviter une confusion entre pays, dataset et compteur individuel.

La première figure EDA vérifie l'ordre de grandeur des consommations. Elle sert à repérer les valeurs extrêmes, les zéros anormaux et les différences d'échelle entre sources avant l'entraînement.

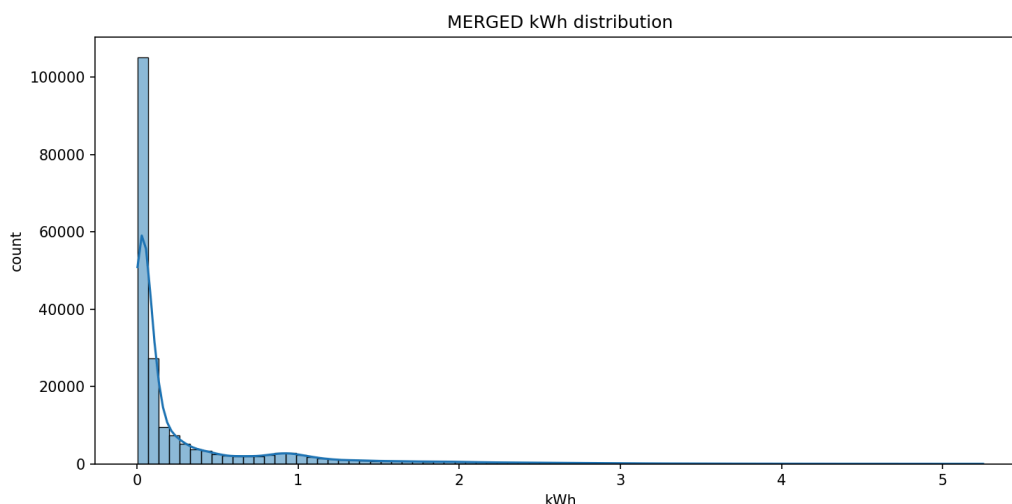


Figure 2: Distribution des consommations dans le jeu préparé. Une distribution très étalée indique que le détecteur doit comparer une mesure à son contexte local plutôt qu'à une moyenne globale unique.

Le profil horaire moyen montre la saisonnalité intra-journalière. C'est cette structure qui justifie les fenêtres temporelles du LSTM et les comparaisons avec les mêmes heures historiques.

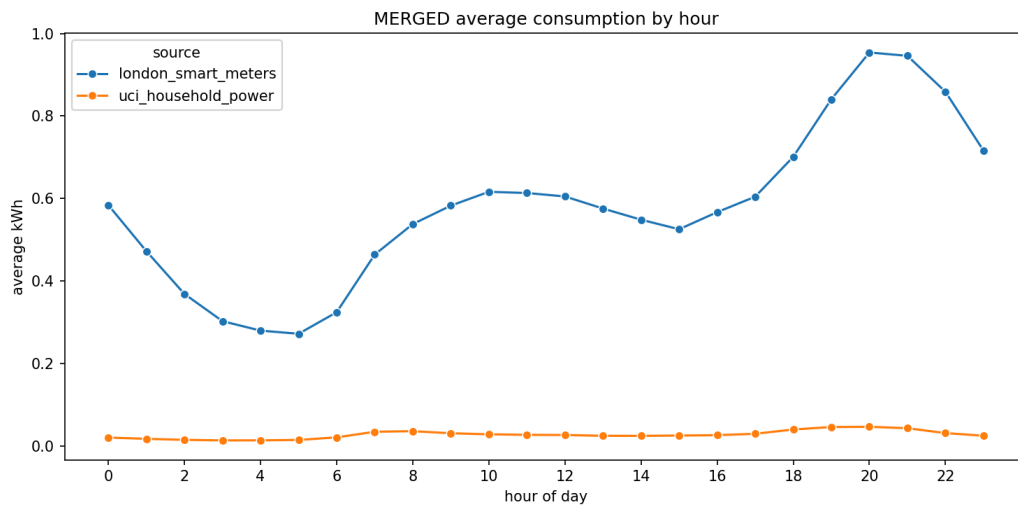


Figure 3: Profil horaire moyen par source de données. Les pics matin/soir et les creux nocturnes donnent le contexte attendu de consommation.

La consommation quotidienne sert à observer les tendances plus lentes. Elle est plus adaptée à Prophet, car ce modèle cherche d'abord une tendance et des composantes saisonnières.

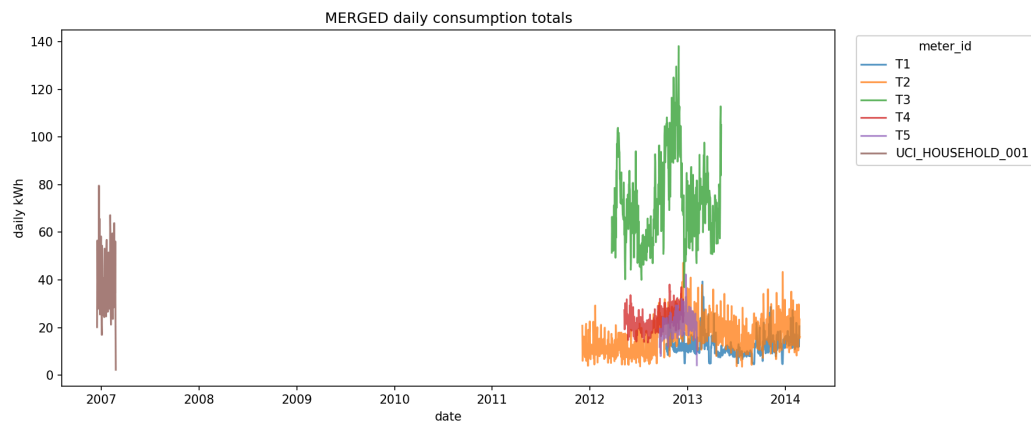


Figure 4: Consommation quotidienne par compteur. Cette vue révèle les variations de niveau et prépare la séparation entre prévision court terme et tendance long terme.

Architecture globale

Le système est organisé autour de deux chemins compatibles. Le chemin streaming reçoit les lectures dans Kafka puis les insère dans TimescaleDB. Le chemin batch charge des CSV préparés dans le même schéma, ce qui permet de réutiliser les mêmes dashboards, agrégats et scripts ML.

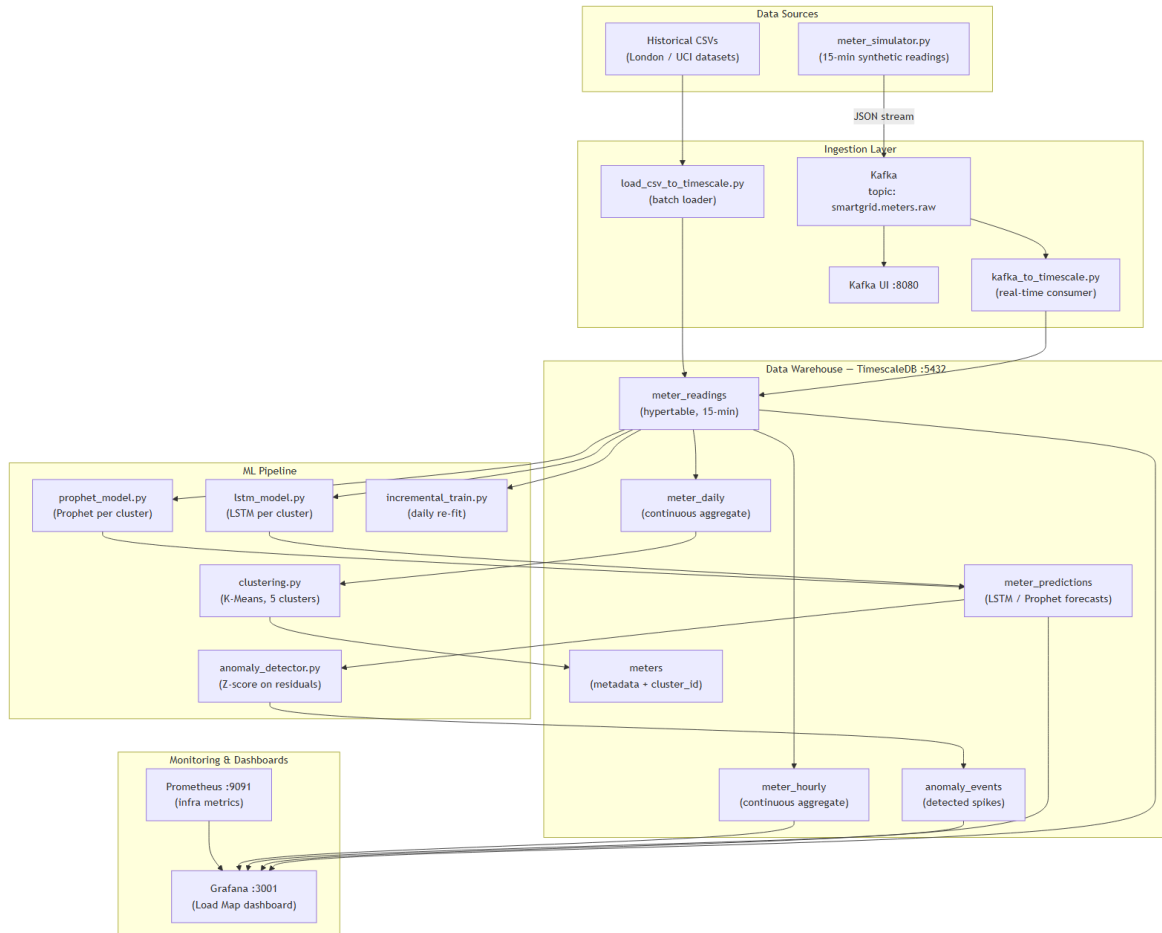


Figure 5: Vue globale technique: les sources alimentent Kafka ou le batch loader, TimescaleDB centralise les séries, les jobs ML écrivent prévisions/anomalies et Grafana expose les résultats.

TimescaleDB contient les tables de base et les agrégats continus nécessaires:

- `meter_readings`: mesures horodatées avec source et label d'anomalie;
- `meter_hourly` et `meter_daily`: agrégats horaires et journaliers;
- `meters`: métadonnées des compteurs;
- `meter_predictions`: sorties de prévision;
- `anomaly_events`: anomalies détectées et sévérité.

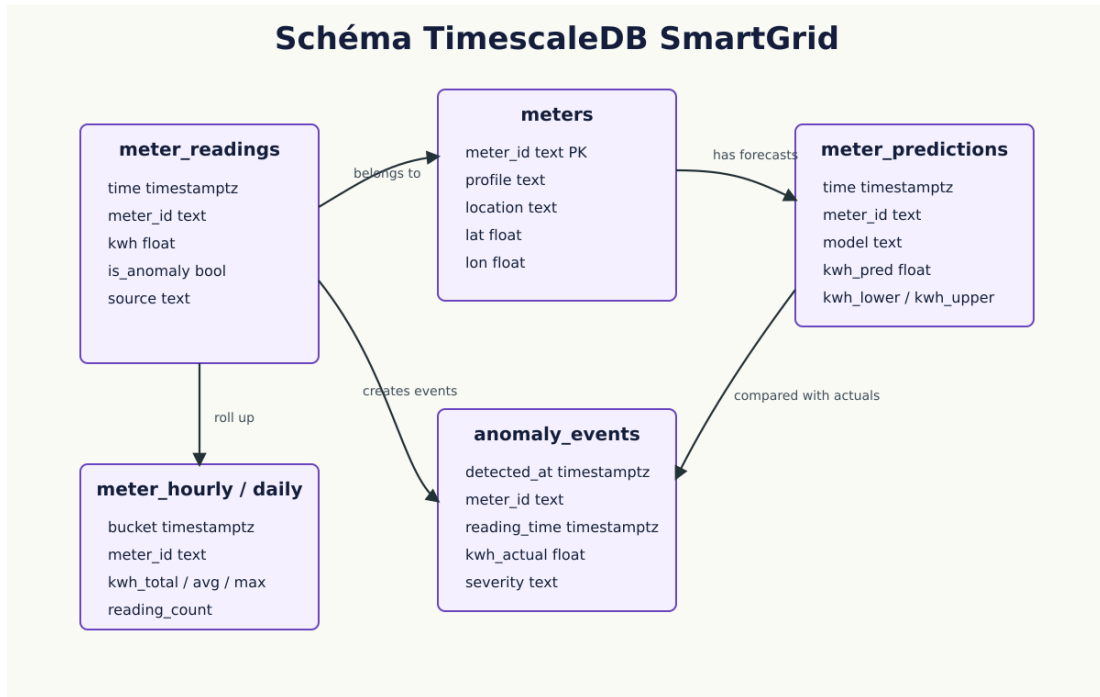


Figure 6: Schéma logique TimescaleDB. Les lectures alimentent les agrégats, les prédictions et les événements d'anomalie consultés par Grafana.

Pipeline Machine Learning

Le pipeline ML suit directement la chronologie d'une série temporelle. Chaque source de données est d'abord nettoyée, convertie en kWh si nécessaire, puis ramenée à une cadence commune. La prévision produit ensuite une valeur attendue; la détection d'anomalies mesure l'écart entre cette valeur attendue et la valeur observée.

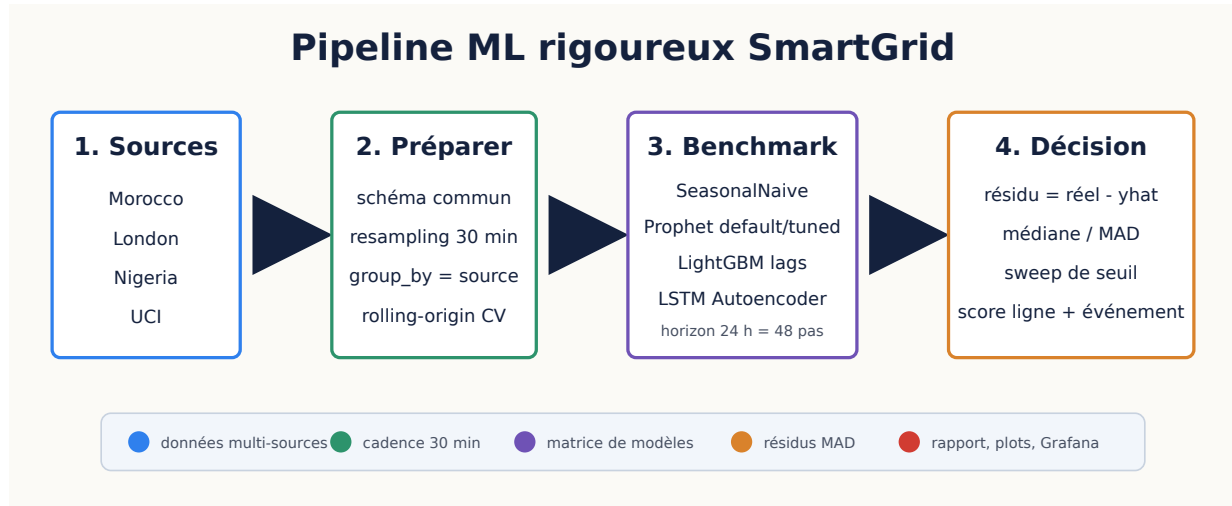


Figure 7: Pipeline ML retenu: sources de données, prétraitement à 30 minutes, benchmark de prévision, calcul du résidu, sélection du seuil MAD et restitution des anomalies dans Grafana.

La chaîne est volontairement séquentielle. Le prétraitement fixe la granularité et le schéma. Les modèles de prévision produisent une valeur attendue au même horizon. La détection compare ensuite le réel au prévu et transforme seulement les écarts significatifs en alertes.

Benchmark de prévision

Le benchmark utilise quatre sources naturelles et cinq folds rolling-origin. La métrique principale est WAPE, plus stable que MAPE lorsque la consommation contient des valeurs proches de zéro. Le minimum industriel est SeasonalNaive; Prophet donne la baseline interprétable; LightGBM teste une approche tabulaire avec lags et variables calendaires.

Modèle	Famille	Statut	WAPE 1 pas	WAPE 24h
SeasonalNaive	forecast	complété	0.0971	0.1259
Prophet default	forecast	complété	0.0594	2.7628
Prophet tuned	forecast	complété	0.0597	1.2224
LightGBM lags	forecast	complété	0.1148	0.0707

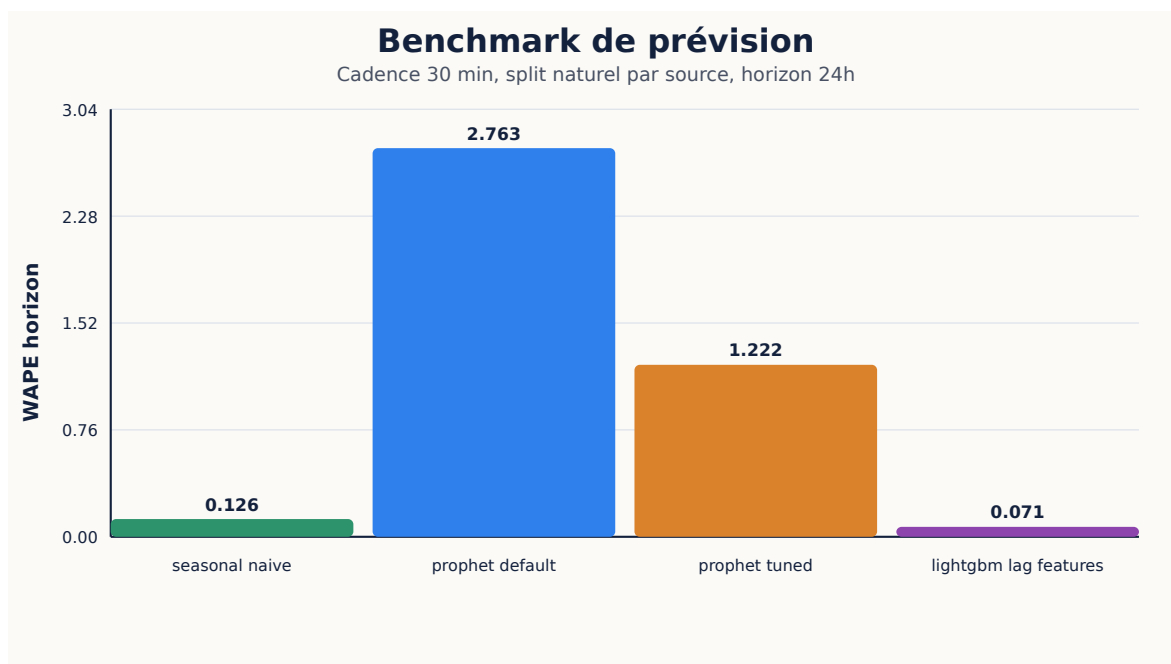


Figure 8: Comparaison WAPE sur horizon 24h. LightGBM obtient la meilleure précision brute sur ce jeu synthétique, tandis que Prophet reste le modèle le plus explicable pour raconter tendance et saisonnalité.

Prévision tendance: Prophet

Prophet modélise une série comme somme de composantes: tendance, saisonnalités et effets calendaires. Dans ce projet, il est utilisé comme baseline interprétable car ses composantes peuvent être expliquées à un opérateur. Le résultat numérique montre aussi sa limite: sur l'horizon 24h synthétique, Prophet extrapole moins bien que LightGBM. Ce n'est pas masqué; le rapport le garde comme modèle principal d'interprétation, pas comme vainqueur automatique du benchmark.

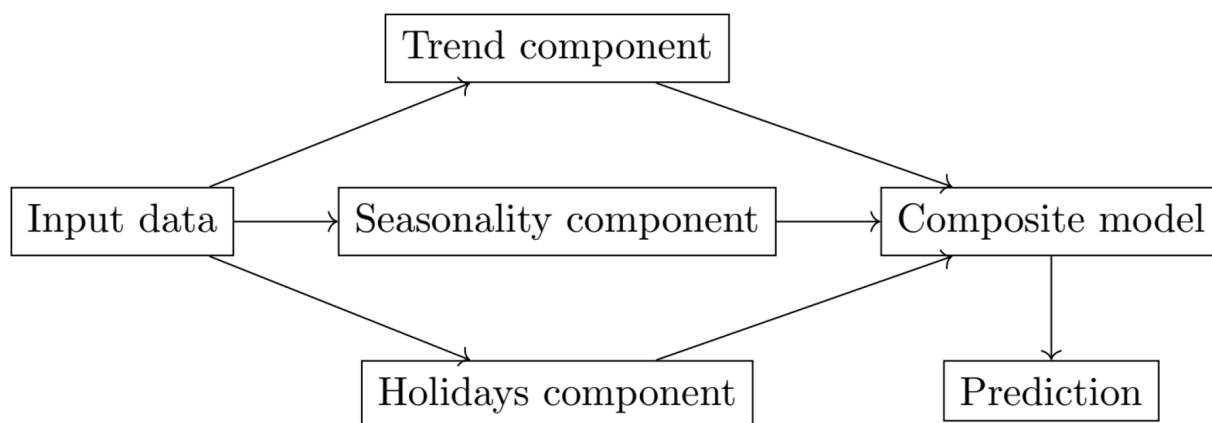


Figure 9: Principe Prophet: la prévision combine tendance et saisonnalités. Ce modèle complète le LSTM en donnant une lecture plus stable du comportement attendu.

Baseline séquentielle: LSTM Autoencoder

Le LSTM Autoencoder n'est pas utilisé comme classifieur supervisé, car aucun label réel d'anomalie industrielle n'est disponible. Il apprend plutôt à reconstruire des fenêtres normales et signale les reconstructions coûteuses. Dans cette expérience, il reste une baseline de recherche: F1 ligne 0.1933, inférieur aux détecteurs résiduels sur les anomalies ponctuelles.

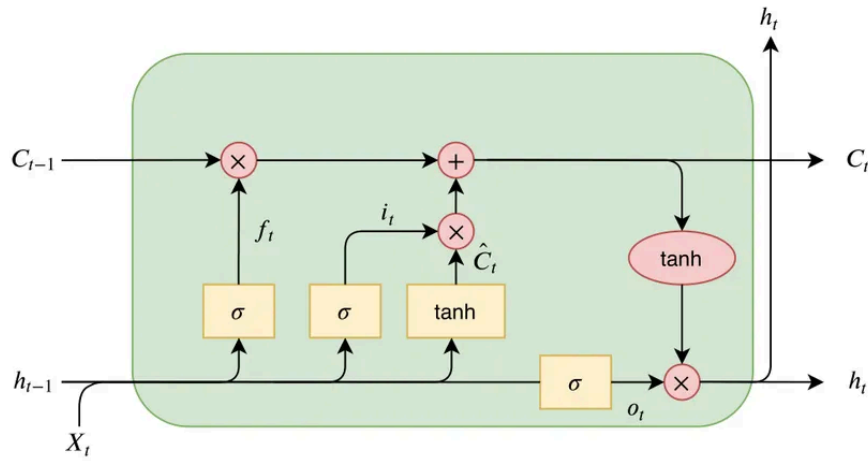


Figure 10: Architecture LSTM: les portes contrôlent la mémoire, l'oubli et la sortie. Cette structure reste utile pour une baseline séquentielle non supervisée.

Démonstration reproductible

La démonstration locale utilise quatre profils synthétiques calibrés pour représenter les sources Morocco, London, Nigeria et UCI. Chaque profil conserve un comportement horaire distinct, ce qui permet de tester la chaîne complète sans dépendre de fichiers bruts volumineux pendant la soutenance. Les sorties ML sont calculées par source avec `freq_minutes = 30`, `horizon_hours = 24`, `horizon_steps = 48` et au moins cinq folds.

Détection d'anomalies et métriques

La détection compare la consommation réelle à la consommation prévue. Le signal utile est le résidu:

$$\text{résidu} = |\text{consommation réelle} - \text{consommation prévue}|$$

Une alerte est créée lorsque ce résidu dépasse un seuil statistique. La démonstration utilise une version robuste basée sur la médiane et la MAD, moins sensible aux valeurs extrêmes qu'une moyenne simple. Le seuil n'est pas une vérité fixe: c'est un réglage métier, sélectionné ici par balayage.

Meilleur WAPE 24h

0.0707

F1 ligne

0.4068

F1 événement

0.5000

Précision événement

0.8571

Les anomalies injectées sont des labels synthétiques vrais, pas des faux positifs. Le protocole inclut quatre familles: point_spike, point_drop, contextual_day_night_swap et trend_drift. Le cas contextuel échange le segment 02:00 avec le segment 14:00: la valeur reste plausible globalement, mais devient mauvaise pour cette heure. Le drift applique une dégradation graduelle $y'_t = y_t + \beta \cdot t$; un détecteur de résidu peut alors ne voir que le début, la fin, ou rien si l'évolution reste trop lisse.

Le seuil MAD retenu par balayage est $MAD = 4.0$, avec précision ligne 0.6316, rappel ligne 0.3000 et F1 ligne 0.4068. Au niveau événement, le même détecteur obtient précision 0.8571, rappel 0.3529 et F1 0.5000. Cette différence est volontaire: le scoring événement exige $k = 5$ flags pour les segments non ponctuels afin de réduire les alertes fragiles; les anomalies ponctuelles gardent $k = 1$.

Seuil MAD	Precision	Recall	F1	TP	FP	FN	TN
1.0	0.5217	0.3000	0.3810	24	22	56	858
2.0	0.5333	0.3000	0.3840	24	21	56	859
3.0	0.5714	0.3000	0.3934	24	18	56	862
4.0	0.6316	0.3000	0.4068	24	14	56	866

Le tableau ne montre que quelques points; le graphe suivant représente le balayage complet de 0.5 à 5.0. Il montre le compromis réel: la précision monte quand le seuil se durcit, mais le rappel chute, surtout pour les anomalies contextuelles et les drifts.

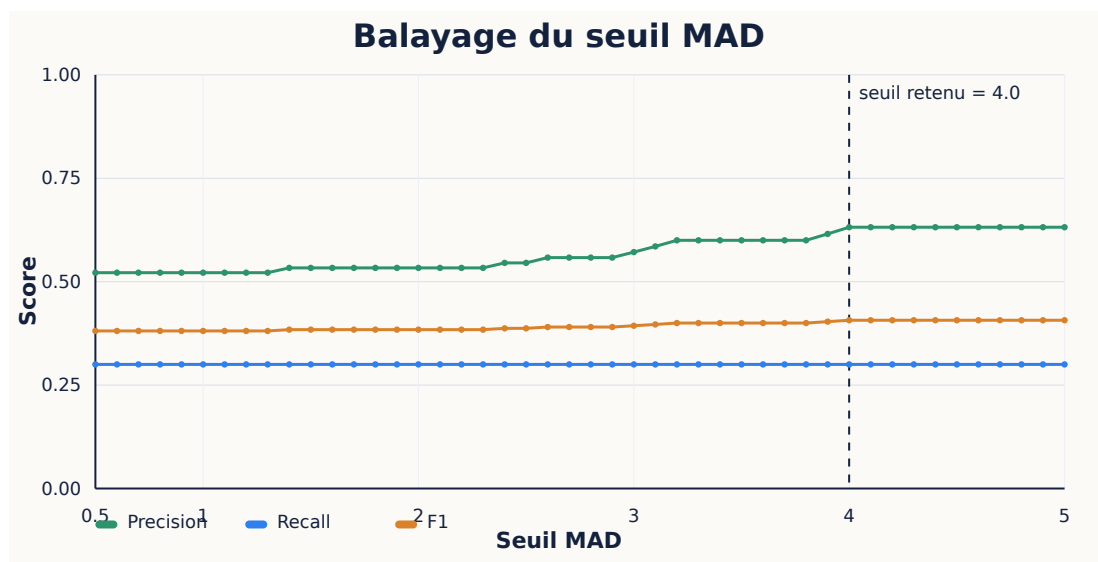


Figure 11: Balayage du seuil MAD sur les résidus LightGBM. Le seuil 4.0 est retenu comme point de fonctionnement pour préserver une précision raisonnable tout en gardant des alertes exploitables.

Détecteur	Précision ligne	Rappel ligne	F1 ligne	TP/FP/FN/TN
SeasonalNaive + MAD	0.3731	0.3247	0.3472	25/42/52/841
Prophet default + MAD	0.4561	0.3377	0.3881	26/31/51/852
Prophet tuned + MAD	0.5102	0.3247	0.3968	25/24/52/859
LightGBM + MAD	0.6316	0.3000	0.4068	24/14/56/866
LSTM Autoencoder	0.1394	0.3152	0.1933	29/179/63/3761

Détecteur	Précision événement	Rappel événement	F1 événement	TP/FP/FN év.
SeasonalNaive + MAD	0.5000	0.4286	0.4615	6/6/8
Prophet default + MAD	0.5556	0.3571	0.4348	5/4/9
Prophet tuned + MAD	0.7143	0.3571	0.4762	5/2/9
LightGBM + MAD	0.8571	0.3529	0.5000	6/1/11
LSTM Autoencoder	0.5000	0.4483	0.4727	13/13/16

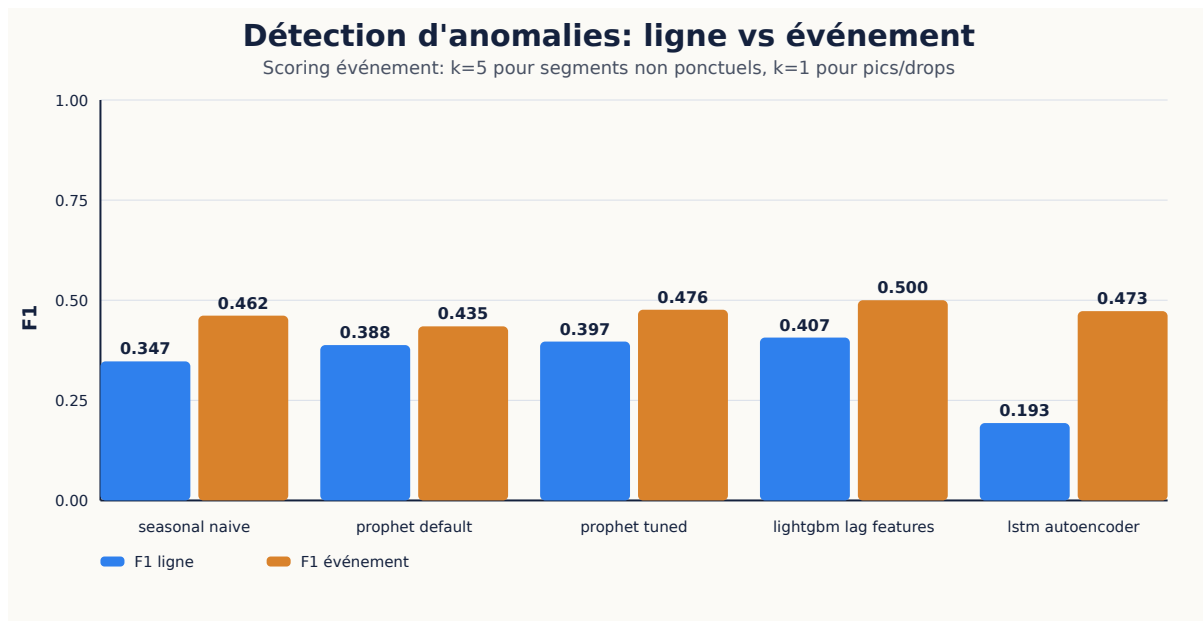


Figure 12: Comparaison F1 ligne vs F1 événement. Le scoring événement pénalise moins les micro-fluctuations isolées, mais révèle clairement que les anomalies contextuelles 02:00/14:00 et les drifts graduels restent difficiles pour un simple résidu MAD.

Pour le meilleur détecteur résiduel LightGBM, les pics et drops sont bien captés au niveau événement: F1 0.9091 pour point_spike et 0.8571 pour point_drop. En revanche, contextual_day_night_swap et trend_drift obtiennent F1 0.0000. C'est un résultat utile: il montre que le benchmark ne récompense pas seulement les anomalies faciles, et qu'un détecteur contextuel explicite ou un modèle de drift est nécessaire pour couvrir ces cas.

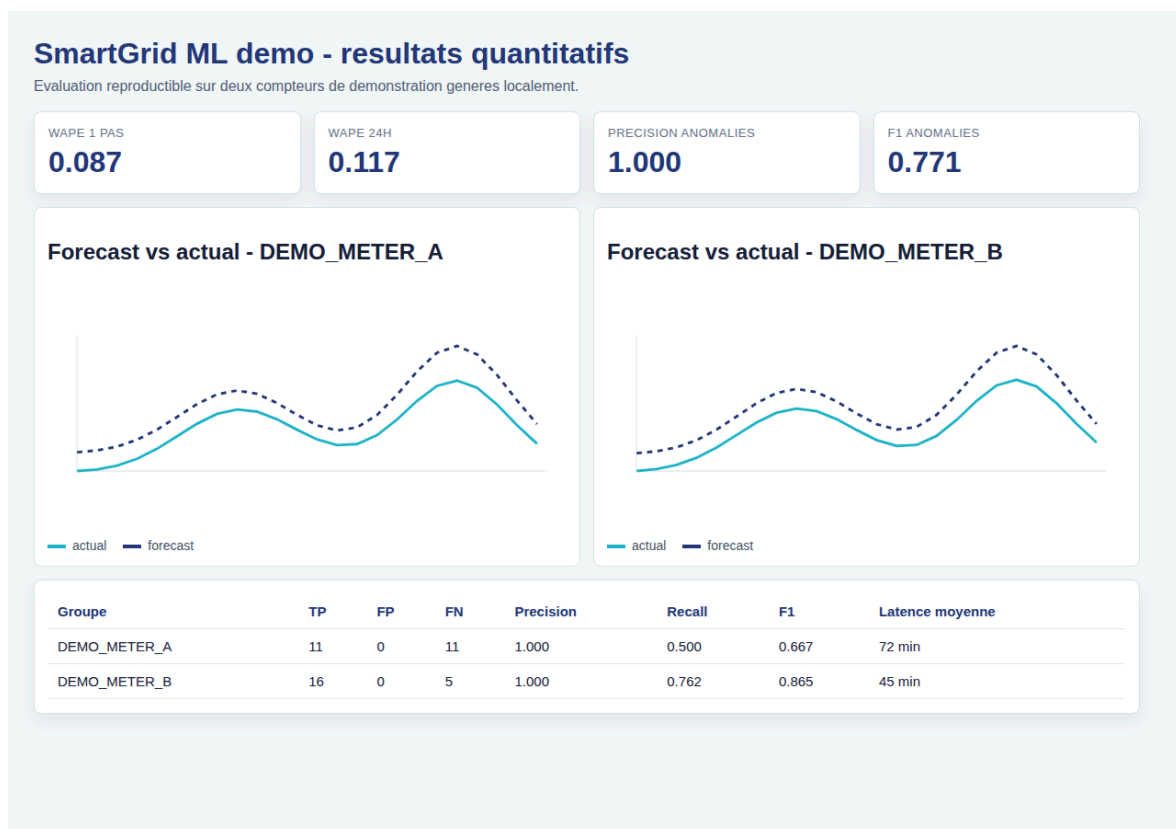


Figure 13: Synthèse locale des métriques ML: WAPE, précision, F1 et comparaison entre consommation réelle et prévue.

Tests automatisés

Les tests unitaires sécurisent les briques critiques:

- erreur WAPE: vérifie le calcul utilisé pour comparer réel et prévision;
- prévision de référence: garantit que le fallback saisonnier produit des sorties exploitables;
- vérité terrain injectée: confirme que les anomalies synthétiques sont bien labellisées;
- pic évident: vérifie que le détecteur MAD signale une rupture nette;
- scoring événement: vérifie que les segments non ponctuels nécessitent plusieurs flags avant d'être comptés positifs;
- table de seuils: contrôle la génération du compromis précision/rappel;
- replay Kafka: vérifie que les messages live conservent la source et peuvent injecter une anomalie.

Ces tests valident la reproductibilité de la démonstration; l'évaluation finale sur données réelles doit ensuite ajouter des backtests par source et une analyse des faux négatifs.

Ingestion, stockage et monitoring

La démonstration locale lance Kafka, Kafka UI, TimescaleDB, Prometheus, Grafana et les exporters Kafka/PostgreSQL. Le bootstrap prépare le topic `smartgrid.meters.raw`, charge les lectures et les sorties ML dans TimescaleDB, rafraîchit les agrégats et vérifie les services.

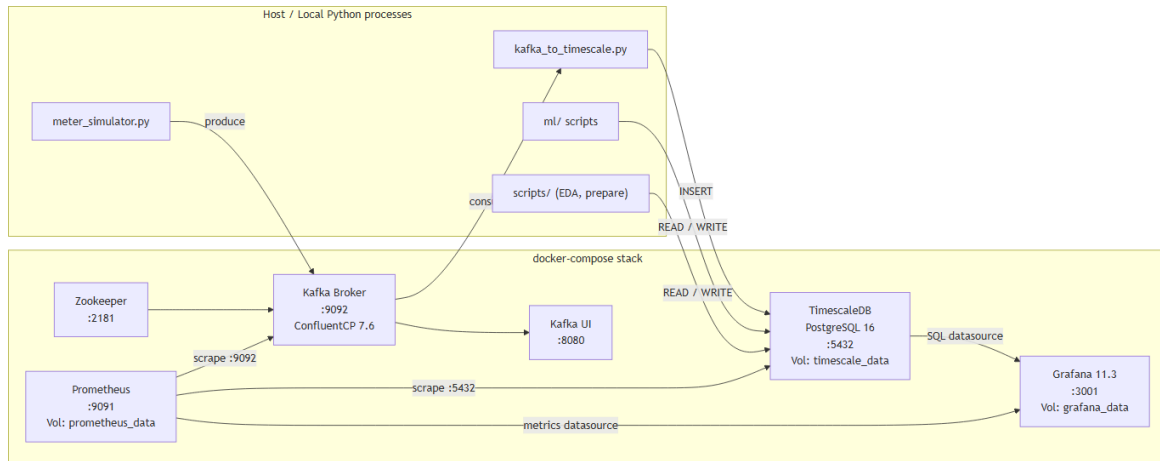


Figure 14: Topologie Docker Compose: Kafka pour l'ingestion, TimescaleDB pour les séries temporelles, Grafana pour la visualisation et Prometheus pour la supervision technique.

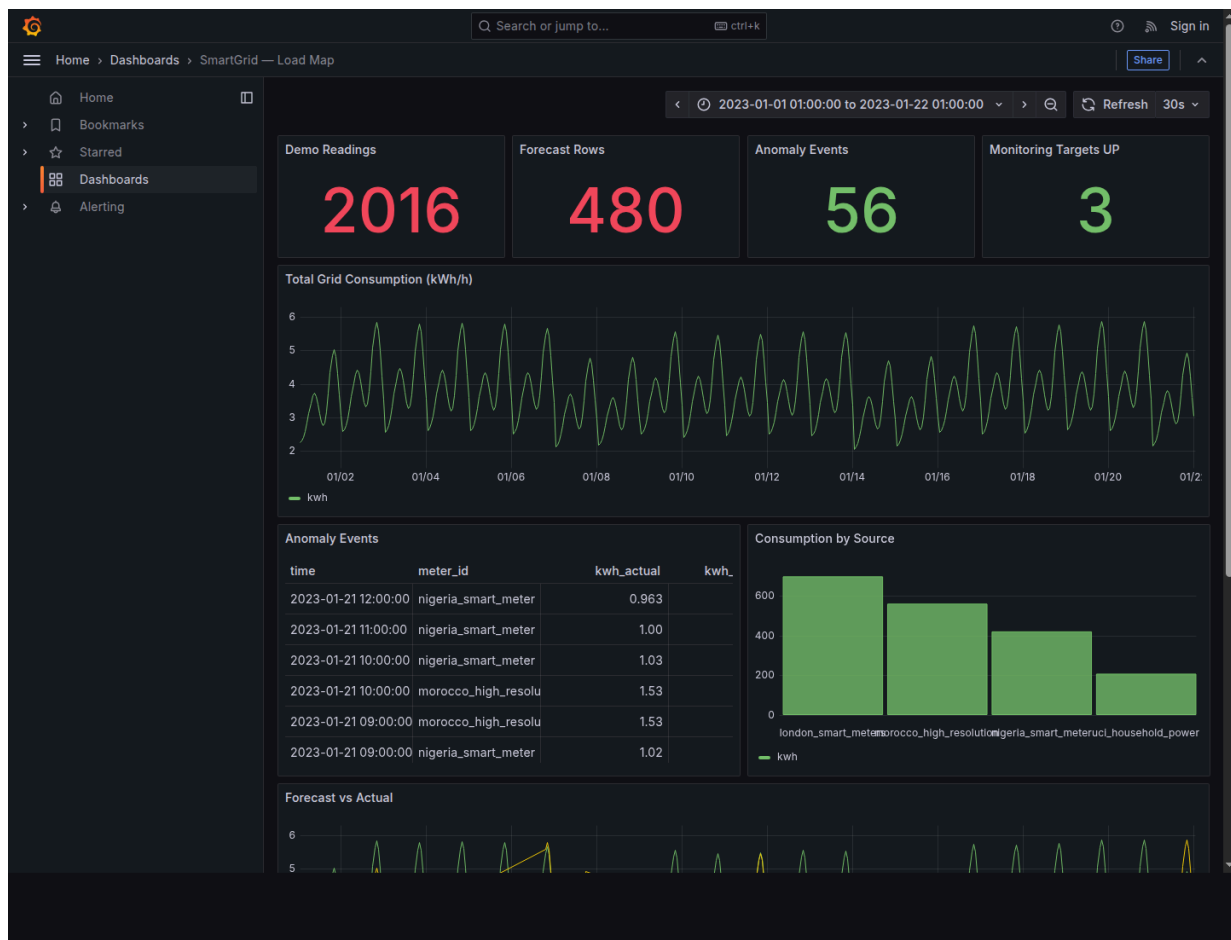


Figure 15: Grafana: dashboard SmartGrid peuplé avec lectures, prévisions, anomalies, consommation par source et indicateur de monitoring.

Grafana est l'écran principal de lecture métier. Il montre les volumes chargés, les prévisions, les anomalies, la consommation agrégée et la comparaison forecast/actual. La vue par source permet de lire séparément Morocco, London, Nigeria et UCI.

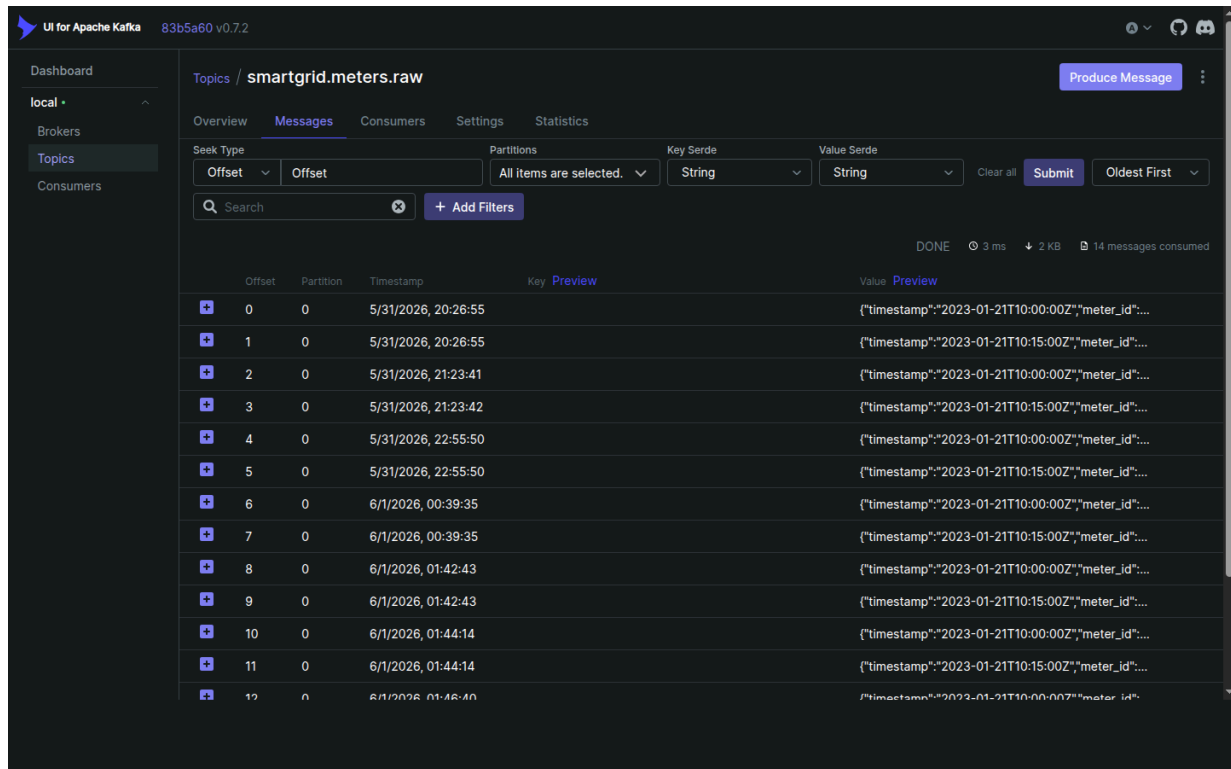


Figure 16: Kafka UI: topic `smartgrid.meters.raw` disponible avec messages de compteur. Ce panneau prouve la couche ingestion et prépare le replay live.

Le replay live suit le standard des démonstrations sans flux industriel réel: on rejoue des profils issus des données préparées, avec horodatages courants, seed déterministe et anomalies injectées. Les messages restent traçables, et la validation compare les anomalies détectées aux labels injectés.

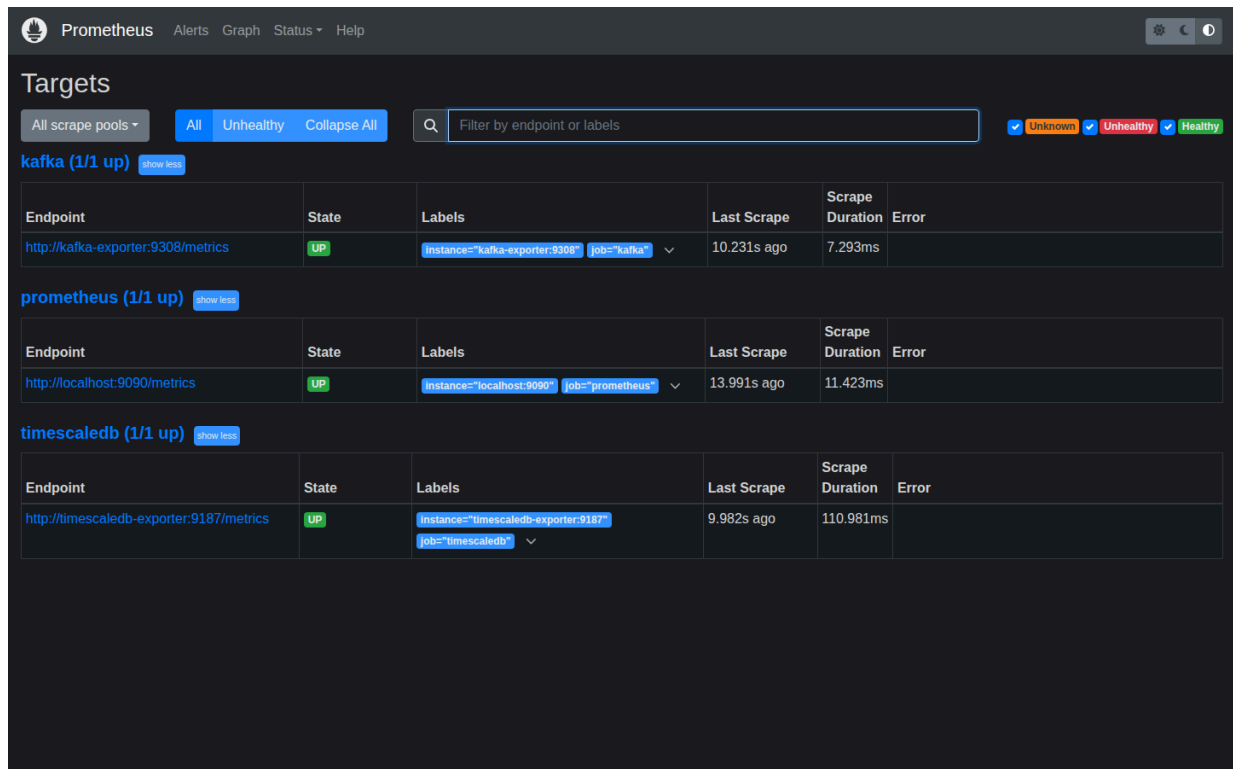


Figure 17: Prometheus: targets Prometheus, Kafka exporter et TimescaleDB exporter en état UP. Prometheus sert à suivre l'infrastructure, pas seulement l'état fonctionnel du dashboard.

La supervision Prometheus doit montrer plus qu'un simple état visuel. Les métriques importantes sont `up`, `scrape_duration_seconds`, les séries exportées par Kafka et les métriques PostgreSQL/TimescaleDB. Ces signaux répondent à la question: le pipeline est-il observable pendant une démonstration ou une exécution prolongée?

Conclusion et perspectives

Le projet valide une chaîne SmartGrid cohérente: normalisation multi-source, prévision, détection d'anomalies, stockage temporel, ingestion Kafka, dashboard Grafana et supervision Prometheus. La partie IA repose sur une idée simple et vérifiable: une anomalie est un écart significatif entre la consommation observée et la consommation prévue pour la même source.

La suite naturelle consiste à remplacer les profils synthétiques de démonstration par les fichiers réels Morocco, London, Nigeria et UCI, puis à refaire les mêmes métriques par source. Cette continuité est importante: le protocole de test reste identique, seuls les profils rejoués et les modèles entraînés changent.